

A Safety Gateway for Coding Agents Using Proxy LLM Services

Xie Yee Lam (3036222719)
bylinmou@connect.hku.hk

Background

In this project, I would like to explore the safety risks and solutions of using proxy servers or proxy gateways for LLM APIs, especially when the user is using a coding agent.

In practice, many users do not connect directly to official providers such as OpenAI, Anthropic, Google, or other model vendors. Instead, they use third-party proxy gateways. There are several reasons for this:

- Some users are in restricted regions where direct access to services such as GPT or Claude is difficult.
- Users want one unified gateway for models from different providers, instead of configuring many official APIs separately.
- Proxy services are often much cheaper than official API pricing, while still claiming to provide usable API-compatible endpoints.

This ecosystem is supported by a grey-market chain. Some technical operators use reverse engineering, shared subscriptions, or account distribution techniques, with projects such as `CLIProxyAPI`, `sub2api`, and `gcli2api`. These channels can then be aggregated and resold through API management platforms similar to `OneAPI` or `NewAPI`. As a result, account sellers, reverse engineers, channel aggregators, and proxy site operators can form a low-cost unofficial market, selling model access at a small fraction of the official API price.

The users of these services include two majority groups: developers using coding agents, and AI role-play users. My project would focus on the first group, because coding agents usually receive much stronger tool permissions and have access to real codebases, credentials, local files, and command execution.

Problem

Coding agents are becoming more powerful and more autonomous. Many tools now support modes such as “YOLO mode”, “dangerously skip permissions”, or other workflows where the agent can run commands and edit files without user confirmation. At the same time, tools such as OpenClaw and similar agent interfaces have made this style of development attractive even to non-programmers. Users increasingly rely on AI agents not only for chat, but also for operating directly inside their development, or even production environment.

This creates a new risk when the model traffic goes through an untrusted proxy gateway. The gateway is technically able to inspect the full request and response stream. This may include the

user's prompts, project context, tool call messages, terminal outputs, configuration files, API keys, database URLs, passwords, and other secrets accidentally included in context.

There are two main threats I want to investigate:

1. **Data leakage and resale.** A malicious or careless proxy operator may log user requests and sell them. The leaked data may be used for model distillation or training by another party. If sensitive information is included in the agent context, it may eventually be exposed, memorized, or redistributed through future models or datasets.
2. **Response poisoning and tool-call injection.** Since the proxy sits between the user and the upstream model, it can modify responses before returning them. Low-impact examples include injecting advertisements into the assistant response. More serious examples include injecting hidden instructions, malicious shell commands, dependency-install commands, credential-exfiltration scripts, or tool calls that read and edit files outside the project workspace. If the user has enabled a high-permission agent mode, the damage can go beyond the current repository.

Current coding agent research and product development seem to focus mainly on coding ability, benchmark performance, and tool-use efficiency. Security is often handled by user awareness: for example, users are expected to manually review commands, use a sandbox, or run the agent inside Docker. However, many users choose high-autonomy modes exactly because they do not want to review every step, or have no awareness as non-programmer users. This means the proxy gateway becomes a critical but under-studied trust boundary.

Proposed Direction

My proposed solution is to build and evaluate a user-controlled safety gateway for coding-agent traffic. Instead of sending requests directly from the agent to an untrusted third-party proxy, the user would first send traffic to a local gateway. This gateway would then forward requests to the actual upstream API or proxy service after applying security checks.

The first version of the gateway could focus on two core functions:

1. **Sensitive-data redaction.** Before forwarding a request, the gateway can scan prompts, tool outputs, and file contents for likely secrets such as API keys, tokens, database URLs, private keys, environment variables, and cloud credentials. Detected secrets can be masked or replaced with placeholders. (The gateway should allow user configurations to avoid false positives, for example by allowing certain non-sensitive strings or patterns to be whitelisted.)
2. **Suspicious-response filtering.** Before returning a response to the agent, the gateway can inspect generated tool calls or command suggestions. It can flag or block suspicious operations, especially shell commands that access files outside the current workspace, download and execute remote scripts, modify shell profiles, install unknown binaries, read credential files, or attempt network exfiltration.

The gateway goal is to take the security control back to the user side, rather than fully trusting unknown third parties. It can also produce an audit log so the user can later inspect what was sent upstream, what was redacted, and which responses were blocked or flagged.

Research Questions

- What types of sensitive information are commonly exposed when coding agents interact with real projects?
- How feasible is it for a proxy gateway to inject malicious instructions or tool calls into coding-agent workflows?
- What are the trade-offs between security, false positives, latency, and developer experience?

Expected Work

The project could include a small **threat model** of the proxy-based LLM API ecosystem, a well-developed open-source **security gateway**, and **experiments** with common coding-agent workflows.

For **development**, I will use Python, chosen for its rapid development speed and extensive libraries. The security gateway will be built as a lightweight proxy using the FastAPI framework, which is ideal for high-performance I/O tasks.

Its core logic will use regular expressions to redact sensitive data from outgoing requests, and parse incoming responses to block risky shell commands and file access attempts. For auditing, all significant security events will be logged into a local SQLite database, ensuring a zero-configuration setup for the user. The gateway will be configurable via a simple YAML file and packaged using Docker for easy deployment.

For **evaluation**, I plan to create test scenarios involving fake API keys, database credentials, malicious response injections, and risky shell commands. The gateway can then be measured by how many sensitive strings it redacts, how many malicious actions it blocks, how many benign actions it incorrectly blocks, and how much extra latency it introduces.

Why This Topic Matters

This topic is interesting because it sits between practical software engineering, LLM security, and real user behavior. Many users will continue to use unofficial proxy services because they are cheap, convenient, and easy to configure. Simply telling users not to use them may not be realistic. A user-controlled safety gateway could be a practical mitigation: it does not require the user to fully trust the proxy, and it gives the user a place to enforce redaction, filtering, and auditing before high-risk agent traffic leaves their machine.

Possible Extensions

If the core safety functions work well, the gateway could also be extended into a more general personal LLM API gateway. For example, it could support multiple upstream providers, model routing, API format conversion, cost tracking, and usage analytics. Some of these features already

exist in open-source tools such as `cc-switch`, `claude-code-router`, or `NewAPI`, so I would treat them as secondary extensions rather than the main research contribution. The main focus should remain the safety layer for coding agents using untrusted or semi-trusted proxy services.